THEORY TRANSFORMATION APP — FULL SPECIFICATION

OVERVIEW

An application implementing **nine formal analysis functions** for transforming, comparing, and analyzing axiomatic systems and conceptual schemes. Accepts both **formal logic** and **plain English** as input.

UI LAYOUT

```
┌──────────────────────────────────────────────────────────────┐
│ FUNCTION 1: Axiom-Set / Theory Transformation               │
│ ┌──────────────────────┐ ┌──────────────────────┐           │
│ │ INPUT          │ │ OUTPUT        │ │          │
│ └──────────────────────┘ └──────────────────────┘           │
│ [ Custom Instructions ..........................................] │
├──────────────────────────────────────────────────────────────┤
│ FUNCTION 2: Schema Equivalence                              │
│ ┌──────────────────────┐ ┌──────────────────────┐           │
│ └──────────────────────┘ └──────────────────────┘           │
│ [ Custom Instructions ..........................................] │
├──────────────────────────────────────────────────────────────┤
│ ... (Functions 3–9 follow identical layout) ...             │
├──────────────────────────────────────────────────────────────┤
│
══════════════════════════════════════════════════════════════│
│ AI CHAT INTERFACE (independent of above functions)          │
│
┌────────────────────────────────────────────────────────────┐│
│ │ [Scrollable Chat Window]                                 ││
│
└────────────────────────────────────────────────────────────┘│
│
┌────────────────────────────────────────────────────────────┐│
│ │ User input...                                            ││
│
└────────────────────────────────────────────────────────────┘│
```

```
│ [Send] [Clear]    LLM: [▼ OpenAI / Anthropic / Grok / DeepSeek ]   │
└──────────────────────────────────────────────────────────────────────────┘
```

FUNCTION ROW STRUCTURE (repeated ×9)

```
┌──────────────────────────────────────────────────────────────────────────┐
│ FUNCTION N: [Name]                                 │
│  ┌──────────────────────┐   ┌──────────────────────┐      │
│  │    INPUT          │   │    OUTPUT        │          │
│  │ (text area)       │   │ (text area)      │          │
│  └──────────────────────┘   └──────────────────────┘      │
│  ┌────────────────────────────────────────────────────────────────────┐ │
│  │ Custom Instructions (text field)                   ││
│  └────────────────────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────────┘
```

THE NINE FUNCTIONS (WITH FULL EXAMPLES)

FUNCTION 1: Axiom-Set / Theory Transformation

What It Does

Takes a user's theory (formal axioms OR plain text) and rewrites it into a new form according to explicit user instructions. It can:

- Change primitives
- Eliminate primitives
- Introduce new primitives
- Restructure axioms
- Invert the conceptual scheme
- Produce an equivalent theory with the same truths

- Explain why the requested transformation is impossible

This is the core engine of the app.

EXAMPLE 1A — Formal Axiomatic System

INPUT (user uploads):

```
Primitives:
  Point(x)
  Between(x, y, z)

Axioms:
1. ∀x∀y∀z [Between(x, y, z) → Point(x) ∧ Point(y) ∧ Point(z)]
2. ∀x∀y [x ≠ y → ∃z Between(x, z, y)]
3. ∀x∀y∀z∀w [(Between(x, y, z) ∧ Between(y, z, w)) → Between(x, y, w)]
```

USER INSTRUCTION: Rewrite this theory using "Line(x, y)" as the sole primitive. Eliminate both Point and Between. Preserve the same models if possible.

OUTPUT (produced by the transformation engine):

```
NEW PRIMITIVE:
  Line(x, y)   // x and y determine a unique line

REWRITTEN AXIOMS (model-preserving version):


1. ∀x∀y [Line(x, y) → x ≠ y]
2. ∀x∀y∀z [Line(x, y) ∧ Line(y, z) → Line(x, z)]
3. ∀x∀y [x ≠ y → ∃z (Line(x, z) ∧ Line(z, y))]


NOTES:
- "Between" is now represented as a composite of two line-relations.
- Axiom 1 ensures all line endpoints are distinct.
- Axiom 2 replaces the transitivity pattern originally carried by "Between".
- The resulting theory is equivalent in all metric-free models.
```

This is a true structural rewrite: different primitives, same geometric content.

EXAMPLE 1B — Plain English Text

INPUT (user uploads):

```
A person is legally an adult if they are at least 18 years old.
People under 18 must obtain parental consent to sign contracts.
People 18 or older may sign contracts freely, without parental approval.
```

USER INSTRUCTION: Rewrite this as a formal rule system using the primitives: `Adult(x)`, `Minor(x)`, `CanSignContract(x)`. Eliminate all age references.

OUTPUT (produced by the engine):

```
PRIMITIVES:
  Adult(x)
  Minor(x)
  CanSignContract(x)

REWRITTEN RULES:

1. ∀x [Adult(x) → ¬Minor(x)]
2. ∀x [Minor(x) → ¬CanSignContract(x)]
3. ∀x [Adult(x) → CanSignContract(x)]

DERIVED DEFINITIONS:
  Adult(x) := legally recognized as 18 or older
  Minor(x) := legally recognized as under 18
```

This shows the engine converting raw prose into a clean axiom set using the desired primitives.

FUNCTION 2: Schema Equivalence

What It Does

Checks whether two different formal systems or representations:

- Use different primitives, or have different syntactic shapes
- But still define the same class of models and therefore express the same truths

The function:

- Takes two inputs (Schema A and Schema B)
- Determines whether they are equivalent
- If not equivalent, explains what differs
- Shows the mapping between corresponding primitives if an equivalence exists

It is a model-theoretic sameness check.

EXAMPLE 2A — Formal Axiomatic Systems

INPUT A (uploaded by user):

```
Schema A — Using "Parent(x, y)"

Primitives:
  Parent(x, y)

Axioms:
1. ∀x∀y [Parent(x, y) → x ≠ y]
2. ∀x∀y∀z [(Parent(x, y) ∧ Parent(y, z)) → Parent(x, z)]
```

INPUT B (uploaded by user):

```
Schema B — Using "Ancestor(x, y)"

Primitives:
  Ancestor(x, y)

Axioms:
1. ∀x∀y [Ancestor(x, y) → x ≠ y]
2. ∀x∀y∀z [(Ancestor(x, y) ∧ Ancestor(y, z)) → Ancestor(x, z)]
3. ∀x∀y [Parent(x, y) → Ancestor(x, y)]   // defined implicitly
```

USER INSTRUCTION: Determine whether Schema A and Schema B are schema-equivalent.

OUTPUT (from the engine):

RESULT: EQUIVALENT UP TO DEFINITIONS

MAPPING:
Parent(x, y) ≈ Ancestor(x, y), since the axioms of each imply the other.

DIFFERENCE:
Schema B is a conservative extension: it includes Ancestor as a defined predicate, but all models of A correspond exactly to models of B with Ancestor interpreted as the transitive closure of Parent.

CONCLUSION:
Schema A and Schema B define the same class of structures.

This shows same model class, different signatures → schema-equivalence.

EXAMPLE 2B — Plain English Text

INPUT A:

A company employs a person if that person works for it full-time or part-time.
A contractor is someone who performs paid labor for a company but is not employed by it.

INPUT B:

A worker is anyone who performs labor for a company.
Workers divide into employees (full-time or part-time) and contractors (not employees).

USER INSTRUCTION: Are these two schemas equivalent? Produce the mapping if they are.

OUTPUT (engine):

```
RESULT: SCHEMA-EQUIVALENT

MAPPING:
  "Employs" in Schema A ↔ "Employee" category in Schema B.
  "Contractor" maps 1:1.
  "Performs paid labor for a company" ↔ "Worker" in Schema B.

NOTES:
  Both schemas divide the domain into:
    (a) employees (full or part-time)
    (b) contractors
  Both generate the same classification of individuals across all interpretations.

CONCLUSION:
  Yes. They are schema-equivalent: different wording, same structure and partition.
```

This shows the function working on ordinary prose, not just math.

FUNCTION 3: Definitional Equivalence

What It Does

Two theories are definitionally equivalent when:

- They may use different primitives
- They may look syntactically different
- But each theory can be translated into the other by giving explicit definitions of the other's primitives
- And these translations are invertible

So: **Each theory can fully define the vocabulary of the other with no loss of information.**

This function:

- Takes two formal systems or texts
- Checks whether each can define the other's primitives
- If yes → returns the bi-directional definitions
- If no → explains the failure

This is deeper than schema equivalence.

- Schema equivalence = same models
- Definitional equivalence = same content down to definitional intertranslatability

---

EXAMPLE 3A — Formal Axiomatic Systems

INPUT A (Theory A):

```
Primitives:
  <  (less-than relation)

Axioms:
1. ∀x ∀y ¬(x < x)
2. ∀x ∀y ∀z [(x < y ∧ y < z) → x < z]
```

INPUT B (Theory B):

```
Primitives:
  Greater(x, y)

Axioms:
1. ∀x ∀y [Greater(x, y) → ¬Greater(y, x)]
2. ∀x ∀y ∀z [(Greater(x, y) ∧ Greater(y, z)) → Greater(x, z)]
```

USER INSTRUCTION: Determine whether Theory A and Theory B are definitionally equivalent. If they are, give the explicit definitions in both directions.

OUTPUT (engine):

```
RESULT: DEFINITIONALLY EQUIVALENT

DEFINITIONS (A → B):
  Greater(x, y) := y < x

DEFINITIONS (B → A):
  x < y := Greater(y, x)

NOTES:
  Each primitive is recoverable from the other by simple reversal.
  All theorems of A map to theorems of B under the translation and vice versa.

CONCLUSION:
  These theories are definitionally equivalent: they are the same ordering theory
  written with opposite primitive relations.
```

This shows: same content, definable translations both ways.

EXAMPLE 3B — Plain English Text

INPUT A (User text A):

```
A person is "eligible" if they satisfy all job requirements.
A requirement is "met" if the person's profile contains the corresponding skill.
```

INPUT B (User text B):

```
A person is "qualified" if every required skill appears in their skill list.
A skill is "required" for a job if the job description lists it.
```

USER INSTRUCTION: Are these definitional equivalents? Provide the definitions if so.

OUTPUT (engine):

```
RESULT: DEFINITIONAL EQUIVALENCE

DEFINITIONS (A → B):
  Qualified(x) := Eligible(x)
  RequiredSkill(s) := Requirement(s)
  MeetsRequirement(x, s) := HasSkill(x, s)

DEFINITIONS (B → A):
  Eligible(x) := Qualified(x)
  Requirement(s) := RequiredSkill(s)
  Met(x, s) := HasSkill(x, s)

NOTES:
  Both schemas assert:
    - A candidate is acceptable exactly when all required skills are in the profile.
  The concepts "eligible" and "qualified" are definitionally identical under direct renaming.

CONCLUSION:
  Yes. These are definitionally equivalent schemas.
```

Here the function demonstrates two-way definability.

FUNCTION 4: Model-Preserving Rewrite

What It Does

Rewrites a theory using a different set of primitives while preserving exactly the same models.

Meaning:

- The rewritten theory must have all and only the models of the original
- The rewrite must express the same truths, just in a different vocabulary
- It is not definitional equivalence; it is a translation into a new primitive basis
- It fails if the target vocabulary is too weak to express the original axioms

---

EXAMPLE 4A — Axiomatic System

INPUT THEORY (original):

```
Primitive: subset ⊆

Axioms:
1. ∀A (A ⊆ A)
2. ∀A∀B [(A ⊆ B ∧ B ⊆ A) → A = B]
3. ∀A∀B∀C [(A ⊆ B ∧ B ⊆ C) → A ⊆ C]
```

CUSTOM INSTRUCTION: Rewrite the theory so that the only primitive is elementhood (∈).

OUTPUT (engine):

```
MODEL-PRESERVING REWRITE (SUCCESSFUL)

Definition used:
  A ⊆ B  :=  ∀x (x ∈ A → x ∈ B)

Rewritten axioms:

1. ∀A ∀x (x ∈ A → x ∈ A)

2. ∀A∀B [ (∀x (x ∈ A → x ∈ B) ∧ ∀x (x ∈ B → x ∈ A)) → A = B ]

3. ∀A∀B∀C [ (∀x (x ∈ A → x ∈ B) ∧ ∀x (x ∈ B → x ∈ C))
            → ∀x (x ∈ A → x ∈ C) ]

MODEL-CHECK:
  The rewritten version has exactly the same models as the original.
  Vocabulary changes; theory extension does not.
```

This shows preservation of semantics under a new primitive basis.

EXAMPLE 4B — Plain Text System

INPUT TEXT (original):

```
A team is effective if every member contributes.
A member contributes if they perform at least one core task.
A core task is any task listed in the role description.
```

CUSTOM INSTRUCTION: Rewrite the system using only the primitive "performs(x, y)". Eliminate the words: "contributes", "core task", "effective".

OUTPUT (engine):

```
MODEL-PRESERVING REWRITE (SUCCESSFUL)

Definitions used:
  Contributes(m) := ∃t performs(m, t)
  CoreTask(t) := role_description_includes(t)
  Effective(team) := ∀m (member_of(m, team) → Contributes(m))

Rewritten system in terms of performs(·,·):

1. A team is effective :=
     ∀m (member_of(m, team) → ∃t performs(m, t))

2. A member contributes :=
     ∃t performs(m, t)

3. A task is a core task :=
     role_description_includes(t)   (this becomes a parameter, not a primitive)

NOTES:
  All semantic content preserved.
  Only the vocabulary has been replaced with the chosen primitive.
```

This demonstrates: same meaning, different language.

FUNCTION 5: Conservative Extension Analysis

What It Does

Checks whether adding new primitives or axioms to a theory:

- Changes none of the old theorems
- Proves nothing new in the original vocabulary
- Does not collapse or strengthen the original theory

If the extension adds expressive convenience but no new consequences about the old concepts, then it is a **conservative extension**.

If new consequences arise even indirectly, the extension is **not conservative**.

EXAMPLE 5A — Axiomatic System

INPUT THEORY (base):

```
Primitives: 0, S, +
Axioms:
1. ∀x x + 0 = x
2. ∀x∀y x + S(y) = S(x + y)
```

PROPOSED EXTENSION:

```
Add primitive: ×
Add axioms:
1. ∀x x × 0 = 0
2. ∀x∀y x × S(y) = (x × y) + x
```

OUTPUT (engine):

```
CONSERVATIVE EXTENSION: YES

REASON:
Multiplication is already definable from addition and successor:
  x × y = repeated_addition(x, y)

No theorem in the language {0, S, +} becomes newly provable.
No contradiction or strengthening of the base theory arises.

CONCLUSION:
The extension adds convenience but does not change the old theory.
```

EXAMPLE 5B — Plain Text Theory

INPUT TEXT (base theory):

```
A project succeeds if all required tasks are completed.
A task is required if it appears in the project scope.
```

EXTENDED THEORY:

```
Add primitive: priority-level(t)
Add axiom: High-priority tasks must be completed first.
```

OUTPUT (engine):

```
CONSERVATIVE EXTENSION: NO

REASON:
The new axiom changes what counts as "project success"
even though the original vocabulary did not mention order or priority.

Original entailment:
  Success = all required tasks completed.

Extended entailment:
  Success = all required tasks completed IN THE REQUIRED ORDER.

This creates new consequences about "success" even in the original terms.

CONCLUSION:
The extension is non-conservative.
It alters the meaning and truth-conditions of existing statements.
```

FUNCTION 6: Compare Conceptual Schemes

What It Does

Identifies and compares the conceptual primitives, derived concepts, and dependency structure of a theory or text. It shows:

- What the theory treats as fundamental
- What concepts depend on which others
- Whether two theories carve the domain differently

- How the "conceptual load-bearing" differs between them

It does not check truth or correctness—only conceptual architecture.

EXAMPLE 6A — Axiomatic System

INPUT THEORY:

```
Primitives:
  parent-of(x,y)
  male(x)
  female(x)

Derived:
  father(x,y) := parent-of(x,y) ∧ male(x)
  mother(x,y) := parent-of(x,y) ∧ female(x)
  sibling(x,y) := ∃z(parent-of(z,x) ∧ parent-of(z,y))
```

OUTPUT (engine):

```
CONCEPTUAL SCHEME ANALYSIS:

PRIMITIVES:
  • parent-of
  • male
  • female

DERIVED NOTIONS:
  • father  — depends on: parent-of, male
  • mother  — depends on: parent-of, female
  • sibling — depends on: parent-of (twice)

DEPENDENCY GRAPH:
  parent-of
    ├── father
    ├── mother
    └── sibling

CONCEPTUAL STRUCTURE:
'parent-of' is the load-bearing primitive. Without it, no kinship relations survive.
Gender predicates are leaf primitives (do not support other primitives directly).
```

EXAMPLE 6B — Plain Text Theory

INPUT TEXT:

```
Leadership is the ability to motivate a group toward a shared objective.
Motivation is produced by trust and clear communication.
Trust requires consistency.
```

OUTPUT (engine):

```
CONCEPTUAL SCHEME ANALYSIS:

PRIMITIVES:
  • leadership
  • motivation
  • trust
  • communication
  • consistency

DERIVED RELATIONS:
  • leadership(x) depends on motivation
  • motivation depends on trust and communication
  • trust depends on consistency

DEPENDENCY PATHS:
  consistency → trust → motivation → leadership
  communication → motivation → leadership

CONCEPTUAL STRUCTURE:
Consistency is the deepest primitive.
Leadership is a surface concept supported by a 3-step dependency chain.
Removing 'consistency' collapses the entire explanatory structure.
```

FUNCTION 7: Ontological Dependence

What It Does

Determines whether a theory can survive the removal of one of its primitives, and what collapses if it is removed.

It outputs:

- Whether the primitive is eliminable or load-bearing
- Which axioms fail without it
- What concepts can no longer be defined
- A collapse severity score (0 = none, 1 = total collapse)

In short: does the theory truly need this primitive, or is it dead weight?

EXAMPLE 7A — Axiomatic System

INPUT:

```
THEORY:
Primitives:
  point
  line
  between(x,y,z)
Axioms:
  1. For any two points, there exists a line containing both.
  2. between(x,y,z) implies x, y, z are distinct.
  3. between(x,y,z) is symmetric in x and z.
  4. ... (order axioms)

TEST: Remove the primitive "between".
```

OUTPUT (engine):

```
ONTOLOGICAL DEPENDENCE TEST: between

RESULT: CRITICAL DEPENDENCE

FAILED AXIOMS:
  • Axiom 2 requires 'between'
  • Axiom 3 requires 'between'
  • All order-related axioms collapse

LOST DEFINITIONS:
  • segment
  • ray
  • interior of angle
  • triangle definition

CONSEQUENCE:
The theory reverts to pure incidence geometry.
Cannot express any order or betweenness facts.
Major theorems are unrecoverable.

DEPENDENCE SCORE: 1.00 (total collapse)

VERDICT:
'between' is indispensable.
```

EXAMPLE 7B — Plain Text Theory

INPUT TEXT:

```
A team succeeds when its members coordinate.
Coordination requires communication.
Communication relies on shared vocabulary.
Shared vocabulary comes from repeated interaction.

TEST: Remove the primitive "communication".
```

OUTPUT (engine):

ONTOLOGICAL DEPENDENCE TEST: communication

RESULT: HIGH DEPENDENCE

DEPENDENCY CHAIN:
repeated interaction → shared vocabulary → communication → coordination → team success

IMPACT OF REMOVAL:
• 'coordination' cannot be characterized
• 'team success' loses its defined mechanism
• shared vocabulary no longer functions as an intermediate concept

THEORY REMAINS?:
A very weak skeleton remains but no explanatory flow.

DEPENDENCE SCORE: 0.83

VERDICT:
'communication' is structurally load-bearing.
Theory becomes vacuous without it.

FUNCTION 8: Generate Alternative Conceptualizations

What It Does

Rebuilds the user's theory using a different ontology:

- Keeps the same truths
- But replaces the primitives
- And rewrites all concepts, relations, and definitions to fit the new conceptual universe

It does **conceptual inversion** or **conceptual re-grounding**:

- If the original theory uses social primitives, it can rewrite them into biological or behavioral primitives
- If the original uses geometric primitives, it can rewrite into algebraic or topological primitives
- If the original uses logical primitives, it can rewrite into structural or relational primitives

The result: the same theory, but expressed using a fundamentally different conceptual scheme.

This is the most "cognitive engineering"–oriented function.

EXAMPLE 8A — Axiomatic System

INPUT:

ORIGINAL THEORY (Temporal Logic):
Primitives:
  before(x,y)
Axioms:
  1. before is transitive.
  2. before is irreflexive.
  3. For any x,y: x=y or before(x,y) or before(y,x).

CUSTOM INSTRUCTIONS:
Reconceptualize time without 'before'. Use 'duration' and 'overlaps' instead.

OUTPUT:

ALTERNATIVE CONCEPTUALIZATION:

NEW PRIMITIVES:
duration(x,y,d) — "the temporal distance between x and y is d"
overlaps(x,y) — "events x and y share some portion in time"

NEW DEFINITIONS:
before(x,y) := ∃d>0: duration(x,y,d) ∧ ¬overlaps(x,y)

REWRITTEN AXIOMS:
1. Transitivity:
If duration(x,y,a) and duration(y,z,b) then duration(x,z,a+b).
2. Irreflexive:
No event has positive duration with itself.
3. Totality:
For any x,y:
either overlaps(x,y),
or ∃d>0: duration(x,y,d),
or ∃d>0: duration(y,x,d).

VERIFICATION:
All original truths are recoverable by definition of 'before'.
The ontology has shifted from ordering → metric + overlap.

EXAMPLE 8B — Plain Text Theory

INPUT TEXT:

A market is efficient if agents act rationally.
Rationality means choosing the option with maximum expected utility.
Utility depends on stable preferences.
Preferences depend on personality traits.

CUSTOM INSTRUCTIONS:
Rewrite the entire theory using behavioral primitives, not psychological ones.

OUTPUT:

ALTERNATIVE CONCEPTUALIZATION:

NEW PRIMITIVES:
- observed-choice-frequency(x,a)
- revealed-superiority(a,b)
- environmental-pressures(e,x,a)

NEW DEFINITIONS:
- "agent chooses option a" := observed-choice-frequency(x,a) is maximal.
- "a is preferred to b" := revealed-superiority(a,b).
- "rationality" := stability of revealed-superiority across contexts.

REWRITTEN THEORY:
1. A market is efficient when observed-choice patterns converge.
2. Convergence occurs when environmental-pressures are uniform.
3. Preference stability becomes:
   revealed-superiority(a,b) persists across environments e.

EFFECT:
Theory no longer references internal psychology.
All explanatory structure is rebuilt from externally observable behavior.
Truth-conditions preserved; ontology replaced.

FUNCTION 9: Identify Representational Biases

What It Does

Detects structural biases built into the user's choice of primitives and axioms.

It answers:

- What does the theory make easy to express?
- What does it make hard or impossible to express?
- What viewpoints or ontologies does the theory privilege?
- What perspectives does the theory exclude or distort?

It returns:

- Privileged structures

- Blind spots
- Representational asymmetries
- Conceptual bottlenecks
- Things the theory treats as "natural," "default," or "fundamental"

In short: what worldview is the formalism sneaking in behind the user's back?

EXAMPLE 9A — Axiomatic System

INPUT:

```
THEORY: Basic Graph Theory
Primitives:
  vertex(x)
  edge(x,y)
Axioms:
  1. edge(x,y) → vertex(x) ∧ vertex(y)
  2. edges are undirected (edge(x,y) ↔ edge(y,x))
  3. no loops (¬edge(x,x))
```

OUTPUT:

REPRESENTATIONAL BIAS ANALYSIS:

PRIVILEGED STRUCTURES:
- Pairwise relationships (2-place relations)
- Undirected symmetry (no notion of direction)
- Node-level ontology (graphs are built only from vertices + edges)

DIFFICULT / IMPOSSIBLE TO EXPRESS:
- Directional influence, causation, flow, hierarchy
- Weighted or probabilistic relations
- Hyperedges (relations involving >2 nodes)
- Node attributes, types, or states

IMPLIED WORLDVIEW:
- Systems consist of homogeneous units connected symmetrically.
- All structure is reducible to pairwise connection.
- No concept of asymmetry, order, or magnitude is "natural" here.

BLIND SPOTS:
- Multi-agent coordination
- Temporal precedence
- Resource flow or dependency
- Typed networks (social vs biological vs informational nodes)

CONCEPTUAL BOTTLENECK:
- Everything is forced into binary, undirected edges.

EXAMPLE 9B — Plain Text Theory

INPUT TEXT:

Leadership is the ability to influence others.
Influence comes from authority and charisma.
Authority arises from formal position.
Charisma arises from personal traits.

OUTPUT:

REPRESENTATIONAL BIAS ANALYSIS:

PRIVILEGED STRUCTURES:
- Individual-level explanation (no group or institutional forces)
- Trait-based causality (leadership = inherent characteristics)
- Formal hierarchy (authority derives only from position)

HARD TO EXPRESS:
- Distributed leadership
- Collective action or team-level processes
- Structural incentives or constraints
- Situational or environmental determinants

IMPLICIT WORLDVIEW / ASSUMPTIONS:
- Leadership resides "inside" individuals, not systems.
- Influence is a function of charisma rather than norms, culture, or design.
- Power originates from titles, not networks or resources.

BLIND SPOTS:
- Power asymmetries not tied to formal role
- Adversarial or coercive influence
- Cultural or ecological factors shaping behavior

CONCEPTUAL BOTTLENECK:
- All explanatory weight sits on psychological traits and position.

SUMMARY TABLE

#	Function	Core Question
1	Axiom-Set / Theory Transformation	"Rewrite this theory with different primitives"
2	Schema Equivalence	"Do these define the same models?"
3	Definitional Equivalence	"Can each define the other's vocabulary?"
4	Model-Preserving Rewrite	"Same models, new primitive basis"
5	Conservative Extension Analysis	"Does adding this change old theorems?"
6	Compare Conceptual Schemes	"What's the dependency structure?"
7	Ontological Dependence	"What breaks if I remove this?"
8	Generate Alternative Conceptualizations	"Same truths, different ontology"

#	Function	Core Question
9	Identify Representational Biases	"What worldview is baked in?"

CHAT INTERFACE (Bottom of Page)

Completely **independent** of the nine function rows.

Component	Description
Chat window	Scrollable message history
Input box	User text entry
Send button	Submit message
Clear button	Reset conversation
LLM selector	Dropdown: OpenAI / Anthropic / Grok / DeepSeek

INPUT TYPES SUPPORTED

All nine functions accept:

- **Formal axiomatic systems** (primitives, axioms, definitions in logical notation)
- **Plain English text** (natural language descriptions of rules, theories, concepts)